

  **gbv / jskos-server**

[Code](#) [Issues 61](#) [Pull requests](#) [Agents](#) [Actions](#) [Security and quality](#) [Insights](#)

[Watch 6](#)   

Web service to access JSKOS data

[coli-conc.gbv.de/api/](#)

 MIT license

 **10 stars**  **5 forks**  **6 watching**  **Branches**  **Activity**  **Custom properties**

 **Tags**

 Public repository

 ...  **10 Branches**  **70 Tags**    [Go to file](#) [Add file ↗](#) [Code](#) 

 **nichtich** 2.5.1 

4d6db3a · last week 

 .github/workflows	Rename master branch to main	3 months ago
 .husky	Replace pre-commit dependency with hu...	2 years ago
 bin	refactor: replace duplicated lookup block ...	2 weeks ago
 config	Ignore .env when testing	2 weeks ago
 docker	change configuration of changes API (#2...	3 months ago
 errors	Change registry object handling	3 months ago
 models	feat: add support mapping sameness ide...	3 weeks ago
 public	Start groundwork of user interface (#248)	2 months ago
 routes	POST /mappings/apply (#270)	2 weeks ago
 services	Fix total number of concepts for empty c...	last week
 test	Fix total number of concepts for empty c...	last week
 utils	Document SSSOM/TSV support and make...	2 weeks ago
 views	Fix total number of concepts for empty c...	last week
 .dockerignore	Add Docker deployment (#105)	6 years ago
 .editorconfig	Initial commit	8 years ago
 .gitignore	Add npm run deps for development	3 weeks ago
 .npmrc	Add Node.js requirements to docs and pa...	4 years ago
 CHANGELOG.md	Update jskos-validate	last week
 LICENSE	Add MIT license	8 years ago
 README.md	Document SSSOM/TSV support and make...	2 weeks ago
 ecosystem.example.json	Adjust pm2 ecosystem file and add it to	3 years ago
 eslint.config.js	Update dependencies	4 months ago
 index.js	Experimental use as module (#224)	3 months ago
 package-lock.json	2.5.1	last week
 package.json	2.5.1	last week
 server.js	fix: handle SSSOM/TSV parse errors inste...	2 weeks ago
 status.schema.json	test: add coverage for /registries CRUD a...	3 months ago

 **README**  MIT license 

JSKOS Server

release [v2.5.1](#) [API](#) [Up](#) [license](#) [MIT](#) [Docker](#) [ghcr.io/gbv/jskos-server](#) [Test](#) [passing](#) [readme style](#) [standard](#)

Web service to access [JSKOS](#) data.

JSKOS Server implements the JSKOS API web service and storage for [JSKOS](#) data such as controlled vocabularies, concepts, and concept mappings.

Table of Contents

- [Install](#)
 - [Requirements](#)
 - [Docker](#)
 - [Configuration](#)
 - [User accounts](#)
 - [Access control](#)
 - [Authentication](#)
 - [Data Import](#)
- [Usage](#)
 - [Run Server](#)
 - [Supplemental Scripts](#)
 - [Use as Module](#)
 - [Development and Testing](#)
- [API](#)
 - [General](#)
 - [GET /status](#)
 - [GET /checkAuth](#)
 - [POST /validate](#)
 - [GET /validate](#)
 - [GET /data](#)
 - [GET /concordances](#)
 - [GET /concordances/:_id](#)
 - [POST /concordances](#)
 - [PUT /concordances/:_id](#)
 - [PATCH /concordances/:_id](#)
 - [DELETE /concordances/:_id](#)
 - [GET /mappings](#)
 - [GET /mappings/suggest](#)
 - [GET /mappings/voc](#)
 - [GET /mappings/infer](#)
 - [GET /mappings/:_id](#)
 - [POST /mappings](#)
 - [PUT /mappings/:_id](#)
 - [PATCH /mappings/:_id](#)
 - [DELETE /mappings/:_id](#)
 - [POST /mappings/apply](#)
 - [GET /voc](#)
 - [POST /voc](#)
 - [PUT /voc](#)
 - [DELETE /voc](#)
 - [GET /voc/top](#)
 - [GET /voc/concepts](#)
 - [DELETE /voc/concepts](#)
 - [GET /voc/suggest](#)
 - [GET /voc/search](#)

- [GET /concepts](#)
- [POST /concepts](#)
- [PUT /concepts](#)
- [DELETE /concepts](#)
- [GET /concepts/narrower](#)
- [GET /concepts/ancestors](#)
- [GET /concepts/suggest](#)
- [GET /concepts/search](#)
- [GET /registries](#)
- [GET /registries/suggest](#)
- [POST /registries](#)
- [PUT /registries](#)
- [PATCH /registries](#)
- [DELETE /registries](#)
- [GET /annotations](#)
- [GET /annotations/:_id](#)
- [POST /annotations](#)
- [PUT /annotations/:_id](#)
- [PATCH /annotations/:_id](#)
- [DELETE /annotations/:_id](#)
- [Change Stream Endpoints](#)
- [Errors](#)
- [Deployment](#)
 - [Notes about depolyment on Ubuntu](#)
 - [Update an instances deployed with PM2](#)
 - [Daily Import](#)
 - [Running Behind a Reverse Proxy](#)
- [Data flow](#)
- [Related works](#)
- [Maintainers](#)
- [Contribute](#)
 - [Publish](#)
- [License](#)

Install

Requirements

You need Node.js 22 or higher and access to a [MongoDB database](#) (minimum v4; v6 or v7 recommended).

To enable optional [Change Stream endpoints](#) the MongoDB database must be configured as a replica set. When using Docker please refer to [our Docker documentation](#) for instructions on setting up a replica set. For a non-Docker setup, start `mongod` with the `--replset` flag, then once connect with the [MongoDB Shell](#) and execute:

```
rs.initiate({ _id: "rs0", members: [{ _id: 0, host: "localhost:27017" }] });
```



If the replica set is initialized, JSKOS Server will detect it at startup (the `replSetGetStatus` command is retried up to `changes.retries` times). If Change Streams [are configured](#) but no replica set was detected, JSKOS Server will log an error during startup but continue running with Change Streams disabled.

Docker

The easiest way to install and use JSKOS Server as stand-alone application is with Docker and Docker Compose. Please refer to [our Docker documentation](#) for more information and instructions.

Configuration

You can customize the application settings via a configuration file. By default, this configuration file resides in `config/config.json`. However, it is possible to adjust this path via the `CONFIG_FILE` environment variable. Note that the given path has to be either absolute (i.e. starting with `/`) or relative to the `config/` folder. The configuration file is validated and JSKOS Server will refuse to start on configuration errors.

Configuration is also affected by environment variable `NODE_ENV` being either `development` (default), `test` (when run with `npm test`), or `production`. In `test` the default config file is `./config.test.json` instead of `config.json`. In `production` HTTPS URIs are forced for entities created on POST requests. Environment variable `CONFIG_FILE` can also be set in `.env` file in the root folder unless `NODE_ENV` is `test`.

Validation of configuration

The provided configuration files (user config and environment config) will be validated with the provided [JSON Schema](#) file under `config/config.schema.json` (public URI: <https://gbv.github.io/jskos-server/status.schema.json>). If validation fails, **JSON Server will refuse to start!** Please check whether your configuration is correct after each change. If there is something wrong, the console output will try to provide you with enough detail to fix the issue.

Function `validateConfig` is exported for [used as module](#):

```
import { validateConfig } from "jskos-server"

try {
  validateConfig(config)
} catch(error) {
  console.error(`Invalid configuration: ${error}`)
}
```



Default configuration

All missing keys will be defaulted from `config/config.default.json`. See [endpoint configuration](#) below for additional settings being merged into the configuration.



```

{
  "verbosity": "warn",
  "baseUrl": null,
  "env": "development",
  "title": "JSKOS Server",
  "version": null,
  "port": 3000,
  "proxies": [],
  "mongo": {
    "user": "",
    "pass": "",
    "host": "127.0.0.1",
    "port": 27017,
    "db": "jskos-server",
    "options": {
      "connectTimeoutMS": 360000,
      "socketTimeoutMS": 360000,
      "heartbeatFrequencyMS": 10000
    }
  },
  "auth": {
    "algorithm": "RS256",
    "key": null
  },
  "schemes": true,
  "concepts": true,
  "mappings": {
    "read": {
      "auth": false
    },
    "create": {
      "auth": true
    },
    "update": {
      "auth": true,
      "crossUser": false
    },
    "delete": {
      "auth": true,
      "crossUser": false
    },
    "fromSchemeWhitelist": null,
    "toSchemeWhitelist": null,
    "cardinality": "1-to-n"
  },
  "concordances": true,
  "registries": {
    "read": {
      "auth": false
    },
    "create": {
      "auth": true
    },
    "update": {
      "auth": true,
      "crossUser": false
    },
    "delete": {
      "auth": true,
      "crossUser": false
    },
    "mixedTypes": false
  },
  "annotations": {
    "read": {
      "auth": false
    },
    "create": {
      "auth": true
    },
    "update": {
      "auth": true,
      "crossUser": false
    },
    "delete": {
      "auth": true,
      "crossUser": false
    },
    "moderatingIdentities": [],
    "mismatchTagVocabulary": null
  },
  "..."
}

```

```

    "changes": false,
    "anonymous": false,
    "identityProviders": null,
    "identityGroups": {},
    "identities": null,
    "ips": null
  }

```

Server configuration

If you are [running jskos-server behind a reverse proxy](#), it is necessary to provide the `baseUrl` key as well as the `proxies` key in your configuration. For example:

```

{
  "baseUrl": "https://coli-conc.gbv.de/api/",
  "proxies": ["123.456.789.101", "234.567.891.011"]
}

```



Endpoint Configuration

With the keys `schemes`, `concepts`, `mappings`, `concordances`, `registries`, and `annotations`, you can configure whether endpoints related to the specific functionality should be available. A minimal configuration file to just server read-only vocabulary and concept information could look like this:

```

{
  "mappings": false,
  "annotations": false,
  "concordances": false
}

```



Available actions for each of these endpoints are `read`, `create`, `update`, and `delete`. By default, all types can be read, while `mappings`, `annotations`, and `registries` can be created, updated, and deleted with authentication.

Explanations for additional options:

- **auth** : Boolean. Can be defined only on actions. Defines whether access will require [authentication via JWT](#). By default `false` for `read`, and `true` for all other actions.
- **crossUser** : Boolean or list of URI strings. Can be defined only on `update` and `delete` actions when `auth` is `true`. Defines whether it is possible to edit an entity from a different user than the authenticated one (`true` = allowed for all users, list = allowed for specified user URIs). `false` by default.
- **anonymous** : Boolean. Can be defined on any level (deeper levels will take the values from higher levels if necessary*). If set, no creator and contributor is saved. `false` by default.
- **cardinality** : String. Can be defined only on type `mappings`. Currently possible values: `1-to-n` (default), `1-to-1`. If `1-to-1` is configured, mappings with multiple concepts in `to` will be rejected.
- **identities** : List of URI strings. Can be defined on any level (deeper levels will take the values from higher levels if necessary*). If set, an action with `auth` set to `true` can only be used by users with an URI given in the list. `null` by default (no restrictions).
- **identityProviders** : List of strings. Can be defined on any level (deeper levels will take the values from higher levels if necessary*). If set, an action can only be used by users who have that identity associated with them. `null` by default (no restrictions).
- **identityGroups** : Object mapping URIs to objects with only field `identities`. Keys of this option can be used in field `identities` elsewhere in the configuration to refer to a group of identities.
- **ips** : List of strings. Strings can be IPv4 addresses (e.g. `127.0.0.1`, `123.234.123.234`) or CIDR ranges (e.g. `192.168.0.1/24`). Can be defined on any level (deeper levels will take the values from higher levels if necessary*). If set, an action can only be used by clients with a whitelisted IP address. `null` by default (no restrictions). Note: An empty array will allow all IPs. Note: This property will be removed for security reasons when accessing [GET /status](#) (meaning that clients will not be able to see the whitelisted IP addresses).
- **fromSchemeWhitelist / toSchemeWhitelist** : Can be defined only on type `mappings`. List of scheme objects that are allowed for `fromScheme` / `toScheme` respectively. `null` allows all schemes.
- **mismatchTagVocabulary** : Can be defined only on type `annotations`. A [JSKOS Concept Schemes](#) object with required property `uri`. When configured, concept URIs belonging to this vocabulary can be used to tag mapping mismatches in mapping annotations. See [below](#) for detailed information about configuration and usage of this feature.

* Only applies to actions `create`, `update`, and `delete`.

Note that any properties not mentioned here are not allowed!

Origin of URIs

This fields are hardcoded in the current version so they don't affect configuration yet!

The `create` field of configuration can have two field that control where URIs of newly created URIs come from:

- `uriBase` : a string or Boolean (false by default). Value `true` is replaced by the `baseUrl` of the server. URIs of newly created items must start with this string.
- `uriOrigin` : where URIs of new items origin from. Default value `external` requires clients to provide an URI. Value `uuid` can be used when `uriBase` is set to generate URIs based on UUIDs.

Registries configuration

Endpoint configuration key `registries` can further contain key `types` to control object types collected in registries. By default it is set to `true` for every type with endpoint enabled in [endpoint configuration](#endpoint configuration). Setting a type to `false` will disallow creation and import of [registry](#) having this field. By default, a registry can only have one type of members. Registry configuration key `mixedTypes` can be set to `true` to allow registries to have multiple member types.

Member types can further the configured with three Boolean keys:

- `uriRequired` (default `true`) whether members must have a valid field `uri`
- `mustExist` (default `false`) whether members must exist in the database (and have the right type)
- `skipInvalid` (default `false`) to filter out members that don't fulfill requirement or `uriRequired` or `mustExist`

Setting `mustExist` to `true` and `uriRequired` to `false` results in an invalid configuration because URIs are required to check whether an item exists.

Example:

```
{
  "types": {
    "schemes": true,
    "concepts": {
      "uriRequired": true,
      "mustExist": false,
      "skipInvalid": false
    },
  },
  "mixedTypes": true
}
```



Change Streams Configuration

[Change Stream Endpoints](#) are only enabled if `changes` is set to `true` or to an object with the following optional keys:

- `retries` (integer, default `20`) How many times to retry the `repSetGetStatus` command while waiting for the replica set to initialise before giving up.
- `interval` (integer, default `5000`) Milliseconds to wait between each retry attempt when checking replica-set status.

Only once the replica set is confirmed will the `/.../changes` endpoints become active, unless MongoDB does is not running with replica set.

Mapping Mismatch Tagging for Negative Assessment Annotations

To differentiate why a mapping was annotated with a negative assessment, a mismatch tagging vocabulary can now be configured under `annotations.mismatchTagVocabulary`. In theory, any vocabulary can be used, but [our instance](#) will use a very small "mismatch" vocabulary available in <https://github.com/gbv/jskos-data/tree/master/mismatch>.

To set up mapping mismatch tagging, add the vocabulary to the configuration:

```
{
  "annotations": {
    "mismatchTagVocabulary": {
      "uri": "https://uri.gbv.de/terminology/mismatch/"
    }
  }
}
```



Currently, the vocabulary and its concepts are required to be imported in the same JSKOS Server instance:

```
# Import vocabulary metadata
npm run import schemes https://raw.githubusercontent.com/gbv/jskos-data/master/mismatch/mismatch-scheme.json
# Reset existing concepts (e.g. if old version has been imported previously)
npm run reset -- -t concepts -s "https://uri.gbv.de/terminology/mismatch/"
# Import vocabulary concepts
npm run import concepts -- --set-api https://raw.githubusercontent.com/gbv/jskos-data/master/mismatch/mismatch-concept
```

After restarting JSKOS Server, mapping mismatch tagging is available for annotations. To add such a tag to an annotation, add a `body` field like this:

```
{
  "motivation": "assessing",
  "bodyValue": "-1",
  "body": [
    {
      "type": "SpecificResource",
      "value": "https://uri.gbv.de/terminology/mismatch/scope",
      "purpose": "tagging"
    }
  ]
}
```

Currently, this is the only supported format, i.e. `body` as an array containing an object with `type` of "SpecificResource", `purpose` of "tagging", and the tag concept's URI as `value`.

To identify whether a JSKOS Server instance supports this kind of tagging, check the `/status` endpoint for the `config.annotations.mismatchTagVocabulary` key.

User accounts

jskos-server does not store user accounts but refers to external identity providers and JSON Web Tokens (JWT) for [authentication](#). Users are identified from field `user` of a valid JWT passed to jskos-server with a request, having the following subfields:

- **uri** primary identity of the user.
- **name** optional name of the user for display (must be a string)
- **identities** optional object mapping names of identity providers to identities, each having field
 - **uri** additional identities of the user
 - **name** optional name of the user for display
 - optional arbitrary fields
- optional arbitrary fields

Optional arbitrary fields don't have semantics in jskos-server and their values are never written into the database but they can be used for extended access control.

So a user can have multiple identities, each being an URI. For example, the following user has ORCID `https://orcid.org/0000-0002-2771-9344` and the fictitious GitHub account `https://github.com/account`. Both can be used interchangeably for [access control](#), but the ORCID is stored (as part of field `creator` or `contributor`) when entities are written into the database.

```
{
  "uri": "https://orcid.org/0000-0002-2771-9344",
  "name": "Sofia",
  "affiliation": "http://example.org/university",
  "identities": {
    "github": {
      "uri": "https://github.com/account",
      "name": "Sofia Coding"
    }
  }
}
```

User names are not unique and not mandatory. Identity and user name can also be passed with query parameters `identity` and `identityName`, respectively. The latter can be set to any string, including the empty string to avoid any user name being written to the database. Query parameter `identity` is ignored if authentication is enabled and its value does not match any of the identities listed in the JWT used for authentication.

User groups are not fully supported yet (see [this issue](#)) but

- identities can be grouped in configuration of access control with `identityGroups`.
- access control can be limited to identity providers with `identityProviders`.

Access control

The rights to `read`, `create`, `update` and `delete` entities via API can be controlled via several configuration settings described above ([data import](#) is not limited by these restrictions):

- Restricted access via `ips` is always applied *in addition* to other settings
- Without [authentication](#) (`auth` set to `false`) the server does not know about user accounts. In this case the `creator` and `contributor` fields of an object can be set without limitations (default) or they are ignored when `anonymous` is set to `true`.
- With authentication an action can be limited to accounts listed in `identities` (if set). Rights to `create`, `update`, and `delete` entities can further depend on two controls:
 - i. value of `creator` and `contributor` of a superordinated object. Concepts always belong to vocabularies via `inScheme` or `topConceptOf` and mappings can belong to concordances via `partOf`.
 - ii. settings of `crossUser` together with value of `creator` and `contributor` of the object

The first control is only checked if it has a superordinated object with `contributor` and/or `creator`. This can only be the case for mappings and concepts. The connection to a superordinated object is checked on both the stored object and its modified value, so moving a mapping from one concordance to another is only allowed if access is granted for both. The authenticated user must be listed as `creator` or `contributor` of the superordinated object to pass this control.

The second control is only checked when the first control cannot be applied and only on authenticated actions `update` or `delete` where `anonymous` is set to `false` (this is the default). With `crossUser` set to `false`, the authenticated user must be listed as `creator` of the stored object. With `crossUser` set to `true` any authenticated user (optionally limited to those listed in `identities`) can `update` or `delete` the object.

For authenticated actions with `anonymous` being `false` creation of a new object will always set its initial `creator` to the authenticated user and `update` of an object will always add the user to `contributor` unless it is already included as `creator` or `contributor`. Further modification of `creator` and `contributor` (removal and addition of entries) is limited to vocabularies and concordance by authenticated users listed as `creator` of the object.

Here are some helpful example presets for configuration of "concordances", "mappings", "annotations", or "registries".

Read-only access (does not make sense for annotations):

```
{
  "read": {
    "auth": false
  }
}
```



Anyone can create, but only logged-in users can update and delete (and only their own items):

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": false
  },
  "update": {
    "auth": true,
    "crossUser": false
  },
  "delete": {
    "auth": true,
    "crossUser": false
  }
}
```



Anyone can create, logged-in users can update (independent of creator), logged-in users can delete their own items:

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": false
  },
  "update": {
    "auth": true,
    "crossUser": true
  },
  "delete": {
    "auth": true,
    "crossUser": false
  }
}
```



Anyone can create, as well as update and delete, independent of creator:

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": false
  },
  "update": {
    "auth": false,
    "crossUser": true
  },
  "delete": {
    "auth": false,
    "crossUser": true
  }
}
```



If write access for concept schemes and/or concepts is necessary, it is recommended that they are secured by only allowing certain users (via `identities`) or only allowing certain IP addresses (via `ips`):

Only user with URI `https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619` can write:

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": true,
    "identities": ["https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619"]
  },
  "update": {
    "auth": true,
    "identities": ["https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619"]
  },
  "delete": {
    "auth": true,
    "identities": ["https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619"]
  }
}
```



Only localhost can write:

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": false,
    "ips": ["127.0.0.1"]
  },
  "update": {
    "auth": false,
    "ips": ["127.0.0.1"]
  },
  "delete": {
    "auth": false,
    "ips": ["127.0.0.1"]
  }
}
```



Note that `auth` is set to `false` because it refers to authentication via JWT. The IP filter is separate from that. An even more secure way would be to use both JWT authentication with an `identities` filter as well as an IP filter.

Only user with URI `https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619` can create, but others can update/delete if they are creator/contributor of an entity:

```
{
  "read": {
    "auth": false
  },
  "create": {
    "auth": true,
    "identities": ["https://coli-conc.gbv.de/login/users/c0c1914a-f9d6-4b92-a624-bf44118b6619"]
  },
  "update": {
    "auth": true
  },
  "delete": {
    "auth": true
  }
}
```



A configuration like this will be used to handle concordances in Cocoda. Only selected accounts will be able to create new concordances, but they will be able to add other accounts as creator/contributor so that those accounts will be able to assign mappings to the concordance and edit mappings that belong to the concordance.

Authentication

It is possible to limit certain actions to authenticated users, indicated by the `auth` option (see [example configurations above](#)). Authorization is performed via JWTs ([JSON Web Tokens](#)). To configure authentication, you need to provide the JWT algorithm and the key/secret in the configuration file, like this:

```
"auth": {
  "algorithm": "RS256",
  "key": "-----BEGIN PUBLIC KEY-----\nmIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA57ZWro0jXYTQ9yujxAu7\ne3k4+JRBAqGdC
}
```



The JWT has to be provided as a Bearer token in the authorization header, e.g. `Authorization: Bearer <token>`. The authentication is designed to be used together with an instance of [login-server](#), but it is also possible to use your own JWTs.

JWT Example

The recommended Node.js library for creating JWTs is [jsonwebtoken](#). Note that for simplicity, we are using the HS256 algorithm which is symmetrical. In most cases, it would be better to use RS256 with a library like [node-rsa](#) instead.

Simple config, restricting the `/mappings` endpoint with authentication:

```
{
  "auth": {
    "algorithm": "HS256",
    "key": "yoursecret"
  },
  "mappings": {
    "read": {
      "auth": true
    }
  }
}
```



Creating a JWT:

```
const jwt = require("jsonwebtoken")
// Payload is an object containing the user object with an URI:
const data = {
  user: { uri: "urn:test:hallo" }
}
// Sign the token with our secret
const token = jwt.sign(data, "yoursecret", {
  algorithm: "HS256",
  expiresIn: "7d" // valid for 7 days
})
```



Using the token in a request (using curl):

```
# Request without header should return ForbiddenAccessError (code 403)
curl localhost:3000/mappings
# Request with header should return JSON data (insert your own token and jskos-server URL of course)
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjpw7InVyaSI6InRlc3Q6aGFsbG8ifSwiaWF0IjoxM
```



If you are the only user that is supposed to be authenticated for your instance of jskos-server, you could in theory use something like this to create a token with a long lifetime and use it for all your requests. Please consider the security implications before doing this though.

Login Server Example

If you have multiple users using your instance of jskos-server, it is recommended to use [login-server](#) for authentication. login-server uses the asymmetrical RS256 algorithm by default and will create a public/private key pair on first launch. The public key will be in `./public.key` and you will need that for the configuration:

```
{
  "auth": {
    "algorithm": "RS256",
    "key": "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA57ZWro0jXYTQ9yuJxAu7\ne3k4+JRBAQc"
  }
}
```



After that, you can use [login-client](#) to interact with your login-server instance and receive JWTs. When using WebSockets, login-server will periodically send a new JWT before the previous one expires. You can then use that to authenticate your requests to jskos-server. (An example on how to use login-client can be found in the [source code of login-server](#).)

For testing your authentication without a full-fledged solution using login-client, you can use <http://localhost:3004/token> (where `localhost:3004` is your instance of login-server) to request a JWT.

Data Import

JSKOS Server provides scripts to import JSKOS data into the database or delete data from the database. Right now, mappings, terminologies (concept schemes), concepts, concordances, and annotations, in JSON (object or array of objects) or [NDJSON](#) format are supported.

Import Script

By default the import script has enabled **bulk import** so invalid entities are filtered out and reported only.

Examples of using the import script:

```
# Create indexes for all types
npm run import -- --indexes
# Import RVK scheme (from coli-conc API)
npm run import -- schemes https://coli-conc.gbv.de/rvk/api/voc
# Import RVK concepts (this will take a while)
npm run import -- concepts https://coli-conc.gbv.de/rvk/data/2019_1/rvko_2019_1.ndjson
# Import coli-conc concordances
npm run import -- concordances https://coli-conc.gbv.de/api/concordances

# Batch import multiple files or URLs
npm run import-batch -- mappings files.txt
# files.txt should contain one file or URL per line with the full path and no escaping.
# You can, for example, store these batch import files in folder `imports` which is ignored in git.
```



Note: If you have concepts in your database, make sure to run `npm run import -- --indexes` at least once. This will make sure all necessary indexes are created. Without this step, the `/concepts/suggest` and `/concepts/search` endpoints will not work.

For more information about the import script, run `npm run import -- --help`.

Reset Script

It is also possible to delete entities from the server via the command line. Running the command will first determine what exactly will be deleted and ask you for confirmation:

```
# Will delete everything from database
npm run reset
# Will delete mappings from database
npm run reset -- -t mappings
# Will delete all concepts that belong to a certain concept scheme URI
npm run reset -- -s http://uri.gbv.de/terminology/rvk/
# Will delete all mappings that belong to a certain concordance URI
npm run reset -- -c https://gbv.github.io/jskos/context.json
# Will delete entities with certain URIs
npm run reset -- http://rvk.uni-regensburg.de/nt/A http://rvk.uni-regensburg.de/nt/B
# Will show help for more information
npm run reset -- --help
```



For scripting, you can use the `yes` command to skip confirmation. **Make sure you know what you're doing!** Example: `yes | npm run reset -- urn:test:uri`.

Specifics of importing concepts

Only the `broader` field will be used during import of concepts: both `ancestors` and `narrower` will be removed and the respective endpoints ([GET /concepts/ancestors](#) and [GET /concepts/narrower](#)) will dynamically rebuild these properties. That means that when converting your data, please normalize it so that the hierarchy is expressed via the `broader` field in JSKOS.

Example scheme (as JSON object) with concepts in a hierarchy (as NDJSON):

```
{
  "uri": "urn:test:scheme",
  "notation": [
    "TEST"
  ],
  "uriPattern": "^urn:test:concept-(.+)$"
}
```



```
{ "topConceptOf": [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-a" }
{ "inScheme":    [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-a.1", "broader": [{ "uri": "urn:test:cc" }], "uri": "urn:test:concept-a.1" }
{ "inScheme":    [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-a.2", "broader": [{ "uri": "urn:test:cc" }], "uri": "urn:test:concept-a.2" }
{ "topConceptOf": [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-b" }
{ "inScheme":    [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-b.1", "broader": [{ "uri": "urn:test:cc" }], "uri": "urn:test:concept-b.1" }
{ "inScheme":    [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-b.1.1", "broader": [{ "uri": "urn:test:cc" }], "uri": "urn:test:concept-b.1.1" }
{ "inScheme":    [{ "uri": "urn:test:scheme" }], "uri": "urn:test:concept-b.1.2", "broader": [{ "uri": "urn:test:cc" }], "uri": "urn:test:concept-b.1.2" }
```



(Note that a notation for the concepts can be omitted because we have defined `uriPattern` on the concept scheme. Also, we don't need to define `inScheme` for concepts with `topConceptOf`.)

Specifics of importing concept schemes

The import script uses the bulk write endpoints to import data. For concept schemes, this means that any existing data for imported schemes will be **overwritten** and replaced with the new data. This includes especially the `created` property which might not exist in your source data and will be set on import if necessary. If you need a consistent `created` date, make sure that your source data already includes this field.

Usage

Run Server

```
# Development server with hot reload and auto reconnect at localhost:3000 (default)
npm run start

# To run the server in production, run this:
NODE_ENV=production node ./server.js
```



Supplemental Scripts

In addition to [data import](#) there are some supplemental scripts that were added to deal with specific situations. These can be called with `npm run script name-of-script`. The following scripts are available:

- `supplementNotationsInMappings`: This will look for mappings where the field `notation` is missing for any of the concepts, and it will attempt to supplement those notations. This only works for vocabularies which are also imported into the same jskos-server instance and where either `uriPattern` or `namespace` are given.

Use as Module

Use as module is experimental. Authentication is ignored on direct access to services!

```
import { validateConfig, createServices } from "jskos-server"

try {
  validateConfig(config)
} catch(error) {
  console.error(`Invalid configuration: ${error}`)
}

const services = createServices(config)

const scheme = await services.scheme.getScheme(schemeUri)
```



Development and Testing

Tests use an ephemeral, in-memory MongoDB server powered by [mongodb-memory-server](#).

```
npm test
```



This will:

1. **Start** a MongoDB (sometimes also a replica-set) entirely in memory.
2. **Connect** Mongoose to that in-memory server.
3. **Create** all JSKOS collections & indexes via services.
4. **Drop** the database before and after each test suite, ensuring full isolation.
5. **Tear down** the in-memory server when the suite completes.

Code coverage can be calculated with `npm run coverage` but numbers should be taken with a grain of salt. By the way, lines of code can be calculated with `npm run loc` (if command line tool `cloc` is installed) and an internal dependency tree (excluding tests) can be generated with `npm run deps`.

You can also start an in-memory MongoDB with local configuration:

```
npm run mongodb          # not verbose
npm run mongodb -- --debug # very verbose
```



And then start jskos-server:

```
npm run start
```



API

General

All API methods stick to the following rules, unless otherwise specified.

Requests

- All URL parameters are optional.
- `POST` / `PUT` / `PATCH` requests require a JSON request body
- Alternatively, `POST` can also receive the following inputs:
 - any kind of JSON stream
 - SSSOM/TSV for [POST /mappings](#), indicated with Content-Type request header `application/sssom+tsv` or `text/tab-separated-values`
 - multipart/form-data with the file in `data`
 - a URL with JSON as `url` in the request params
 - Note: The `type` request param might be required (either `json`, `ndjson`, or `multipart`)
- All `GET` endpoints returning a certain type of JSKOS data offer the `properties=[list]` parameter, with `[list]` being a comma-separated list of properties.
 - All JSKOS types allow removing properties by prefixing the property with `-`. All following properties in the list will also be removed.
 - For concepts and mappings, the property `annotations` can be specified to add all annotations in the database for a certain item.
 - For concepts, the properties `narrower` and `ancestors` can be specified to add narrower/ancestor concepts to a certain concept.
 - Specifying a `*` adds all available properties.
 - Example: `properties=*, -narrower, notation` will add properties `annotations` and `ancestors`, and remove the `notation` property from all return items.
 - Properties can be explicitly re-added by prefixing them with `+`, e.g. `properties=-from,to,+from` will only remove the `to` property.
 - Note that the `+` sign has to be properly encoded as `%2B`, otherwise it will be interpreted as a space.
- All `GET` endpoints (except for `/status` and those with `:_id`) offer pagination via `limit=[number]` (default: 100) and `offset=[number]` (default: 0) parameters. In the response, there will be a `Link` header like described in the [GitHub API documentation](#), as well as a `X-Total-Count` header containing the total number of results.

Write access

- `POST` / `PUT` / `PATCH` / `DELETE` requests require [authentication](#) via a JWT from [login-server](#) in the header. Exception: Authentication for certain actions on certain endpoints can be disabled (see [configuration](#)).
- `PUT` / `PATCH` / `DELETE` requests are required to come from the owner of the entity that is being modified.
- `POST` / `PUT` / `PATCH` endpoints will override `creator` and `contributor` of submitted objects (see [this comment](#) for more details)
- `PATCH` request bodies are merged on the top level, so it's enough to include object properties to be modified. To remove a top-level property, set it to `null`.

Responses

- `GET` requests will return code 200 on success.
- `POST` requests will return code 200 or 201 on success.
- `DELETE` requests will return code 204 on success.
- For possible error responses, see [Errors](#).

GET /status

Returns a status object.

There is a [JSON Schema](#) for the format of this endpoint. It is available under `/status.schema.json` for every jskos-server installation (starting from version 1.0.0). The most recent schema can be accessed here: <https://gbv.github.io/jskos-server/status.schema.json>

Note that certain properties from the actual configuration will not be shown in the result for `/status`:

- `verbosity`
- `port`
- `mongo`
- `namespace`

- `proxies`
- `ips` (including inside of actions)
- `auth.key` if a symmetrical algorithm is used (HS256, HS384, HS512)
- **Success Response**



```

{
  "config": {
    "env": "development",
    "baseUrl": "http://localhost:3000/",
    "version": "1.1",
    "auth": {
      "algorithm": "RS256",
      "key": null
    },
    "schemes": {
      "read": {
        "auth": false
      }
    },
    "concepts": {
      "read": {
        "auth": false
      }
    },
    "mappings": {
      "read": {
        "auth": false
      },
      "create": {
        "auth": true
      },
      "update": {
        "auth": true,
        "crossUser": false
      },
      "delete": {
        "auth": true,
        "crossUser": false
      },
      "fromSchemeWhitelist": null,
      "toSchemeWhitelist": null,
      "anonymous": false,
      "cardinality": "1-to-n"
    },
    "concordances": {
      "read": {
        "auth": false
      }
    },
    "annotations": {
      "read": {
        "auth": false
      },
      "create": {
        "auth": true
      },
      "update": {
        "auth": true,
        "crossUser": false
      },
      "delete": {
        "auth": true,
        "crossUser": false
      },
      "registries": {
        "read": {
          "auth": false
        },
        "create": {
          "auth": true
        },
        "update": {
          "auth": true,
          "crossUser": false
        },
        "delete": {
          "auth": true,
          "crossUser": false
        }
      },
      "mismatchTagVocabulary": {
        "uri": "https://uri.gbv.de/terminology/mismatch/",
        "API": [
          {
            "type": "http://bartoc.org/api-type/jskos",
            "url": "http://localhost:3000/"
          }
        ]
      }
    }
  },
  "mismatchTagVocabulary": {
    "uri": "https://uri.gbv.de/terminology/mismatch/",
    "API": [
      {
        "type": "http://bartoc.org/api-type/jskos",
        "url": "http://localhost:3000/"
      }
    ]
  }
}

```

```

    }
  },
  "identityProviders": null,
  "identities": null
},
"data": "http://localhost:3000/data",
"schemes": "http://localhost:3000/voc",
"top": "http://localhost:3000/voc/top",
"voc-search": "http://localhost:3000/voc/search",
"voc-suggest": "http://localhost:3000/voc/suggest",
"voc-concepts": "http://localhost:3000/voc/concepts",
"concepts": "http://localhost:3000/concepts",
"narrower": "http://localhost:3000/concepts/narrower",
"ancestors": "http://localhost:3000/concepts/ancestors",
"search": "http://localhost:3000/concepts/search",
"suggest": "http://localhost:3000/concepts/suggest",
"mappings": "http://localhost:3000/mappings",
"concordances": "http://localhost:3000/concordances",
"annotations": "http://localhost:3000/annotations",
"registries": "http://localhost:3000/registries",
"types": null,
"validate": "http://localhost:3000/validate",
"ok": 1
}

```

- **Error Response**

In case of an error, for instance a failed database connection, the value of response property `ok` is set to `0`.

GET /checkAuth

Endpoint to check whether a user is authorized (see [user accounts](#) and [access control](#)). If `type` or `action` are not set, it will use `identities`, `identityProviders`, and `identityGroups` that are defined directly under `config`.

- **URL Params**

`type=[type]` one of "schemes", "concepts", "mappings", "concordances", "registries", "annotations" (optional)

`action=[action]` one of "read", "create", "update", "delete" (optional)

`identity` optional identity URI (must be valid if authentication is required)

`identityName` optional identity name

- **Success Response**

JSON Object with [user data](#) that can be written into the database (`uri` and/or `name`).

POST /validate

Endpoint to validate JSKOS objects via [jskos-validate](#).

- **URL Params**

`type=[type]` a [JSKOS object type](#) that all objects must have (optional)

`unknownFields=[boolean]` with `1` or `true` to allow unknown fields inside objects (by default, unknown fields do not pass validation)

`knownSchemes=[boolean]` with `1` or `true` to use concept schemes available in this jskos-server instance for validation of concepts. Implies `type=concept` and all concept must reference a known concept scheme via `inScheme`.

If neither `type` nor `knownSchemes` are specified, concept schemes in the data to be validated can be used to validate following concepts in the same request array (see last example below).

- **Success Response**

Array with the JSON response provided by [jskos-validate](#). The indices of the array correspond to the order of the given data. An element is `true` when the object passed validation, or an array of errors when the object failed validation. Data format of error objects may change in future versions but there is always at least field `message`.

- **Sample Call**

In the following example, an empty object is validated. Since no `type` is specified, it is validated as a Resource which does not have required field names and therefore passes validation.

```
curl -X POST "https://coli-conc.gbv.de/dev-api/validate" -H 'Content-Type: application/json' -d '{}'
```



```
[
  true
]
```



In the following example, the same call is given, but the parameter `type` is set to `mapping`. Mappings require the fields `from` and `to`, therefore the empty object fails validation and errors are returned.

```
curl -X POST "https://coli-conc.gbv.de/dev-api/validate?type=mapping" -H 'Content-Type: application/json' -d '{}'
```



```
[
  [
    {
      "instancePath": "",
      "schemaPath": "#/required",
      "keyword": "required",
      "params": {
        "missingProperty": "from"
      },
      "message": "must have required property 'from'"
    },
    {
      "instancePath": "",
      "schemaPath": "#/required",
      "keyword": "required",
      "params": {
        "missingProperty": "to"
      },
      "message": "must have required property 'to'"
    }
  ]
]
```



In this example, an array of mixed typed objects is validated (given in file `example.json`):

```
[
  {
    "type": [ "http://www.w3.org/2004/02/skos/core#ConceptScheme" ],
    "uri": "http://example.org/voc",
    "notationPattern": "[a-z]+"
  },
  {
    "type": [ "http://www.w3.org/2004/02/skos/core#Concept" ],
    "uri": "http://example.org/1",
    "notation": [ "abc" ],
    "inScheme": [ { "uri": "http://example.org/voc" } ]
  },
  {
    "type": [ "http://www.w3.org/2004/02/skos/core#Concept" ],
    "uri": "http://example.org/2",
    "notation": [ "123" ],
    "inScheme": [ { "uri": "http://example.org/voc" } ]
  }
]
```



The first object is a concept scheme with `notationPattern`. Since the other two elements are concepts of that concept scheme (see `inScheme`), the concepts must additionally pass tests related to URI or notation patterns of the given schemes. Since the last concept has a notation that does not match the pattern, it fails the validation. Note that only object with appropriate `type` field are included in this additional part of validation.

```
curl -X POST "https://coli-conc.gbv.de/dev-api/validate" -H 'Content-Type: application/json' -d @example.json
```



```
[
  true,
  true,
  [
    {
      "message": "concept notation 123 does not match [a-z]+"
    }
  ]
]
```



GET /validate

Same as [POST /validate](#) but JSKOS data to be validated is passed via URL.

- **URL Params**

`url=[url]` URL to load JSKOS data from

`type=[type]` see [POST /validate](#)

`unknownFields=[boolean]` see [POST /validate](#)

`knownSchemes=[boolean]` see [POST /validate](#)

GET /data

Returns data for a certain URI or URIs. Can return concept schemes, concepts, concordances, mappings, annotations, and registries. This endpoint does not offer pagination via `limit` and `offset`. It will always return all results. Furthermore, there is no certain order to the result set (but it should be consistent across requests). If a certain type of data requires authentication and the user is not authenticated, that type of data will simply not be returned.

Note: As of version 2.0, this endpoint was adjusted to return all types of items that are available in the database, instead of just concepts and concept schemes. The additional parameters, apart from `uri`, were also removed. For the previous behavior (only without returning concept schemes), see [GET /concepts](#).

- **URL Params**

`uri=[uri]` URIs for JSKOS items separated by `|` (annotations, despite using `id` instead of `uri`, can also be queried here)

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors`, `narrower`, and `annotations`)

- **Success Response**

JSON array of [JSKOS Items](#)

GET /concordances

Lists all concordances for mappings.

- **URL Params**

`uri=[uri]` URIs for concordances separated by `|`

`fromScheme=[uri|notation]` only show concordances from concept scheme (URI or notation) (separated by `|`)

`toScheme=[uri|notation]` only show concordances to concept scheme (URI or notation) (separated by `|`)

`creator=[creator]` only show concordances from creator (separated by `|`)

`mode=[mode]` specify the mode for the parameters above, one of `and` (default) and `or`

`download=[type]` returns the whole result as a download (available types are `json` and `ndjson`), ignores `limit` and `offset`

- **Success Response**

JSON array of [JSKOS Concordances](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/concordances?limit=1
```



```
[
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "creator": [
      {
        "prefLabel": {
          "de": "VZG"
        }
      }
    ],
    "distributions": [
      {
        "download": "https://coli-conc.gbv.de/api/mappings?partOf=http://coli-conc.gbv.de/concordances/ddc_rvk_re",
        "format": "http://format.gbv.de/jskos",
        "mimetype": "application/x-ndjson; charset=utf-8"
      }
    ],
    "extent": "2267",
    "fromScheme": {
      "notation": [
        "DDC"
      ],
      "uri": "http://bartoc.org/en/node/241"
    },
    "notation": [
      "ddc_rvk_recht"
    ],
    "scopeNote": {
      "de": [
        "Recht"
      ]
    },
    "toScheme": {
      "notation": [
        "RVK"
      ],
      "uri": "http://bartoc.org/en/node/533"
    },
    "type": [
      "http://rdfs.org/ns/void#Linkset"
    ],
    "uri": "http://coli-conc.gbv.de/concordances/ddc_rvk_recht"
  }
]
```

GET /concordances/:_id

Returns a specific concordance.

- **URL Params**

None

- **Success Response**

JSKOS object for concordance.

- **Error Response**

If no concordance with `_id` could be found, it will return a 404 not found error.

POST /concordances

Saves one or more concordances in the database. Note that `fromScheme` and `toScheme` must be supported by the jskos-server instance.

- **URL Params**

None

- **Success Response**

JSKOS Concordance object(s) as were saved in the database.

- **Error Response**

When a single concordance is provided, an error can be returned if there's something wrong with it (see [errors](#)). When multiple concordances are provided, the first error will be returned.

PUT /concordances/:_id

Overwrites a concordance in the database.

- **Success Reponse**

JSKOS Concordance object as it was saved in the database.

Note that any changes to the `uri`, `notation`, `fromScheme`, `toScheme`, `extent`, `distributions`, and `created` properties will be ignored. (No error will be thrown in this case.)

PATCH /concordances/:_id

Adjusts a concordance in the database.

- **Success Reponse**

JSKOS Concordance object as it was saved in the database.

Note that changes to the properties `uri`, `notation`, `fromScheme`, `toScheme`, `created`, `extent`, and `distributions` are currently not allowed and will result in an [InvalidBodyError](#).

DELETE /concordances/:_id

Deletes a concordance from the database.

- **Success Reponse**

Status 204, no content.

Note that only concordances which have no mappings associated can be deleted.

GET /mappings

Returns an array of mappings. Each mapping has a property `uri` under which the specific mapping can be accessed.

- **URL Params**

`identifier=[identifier1|identifier2|...]` specify mapping identifiers separated by `|`

`from=[uriOrNotation1|uriOrNotation2|...]` specify the source URI or notation (truncated search possible by appending a `*`, multiple URIs/notations separated by `|`)

`fromScheme=[uriOrNotation1|uriOrNotation2|...]` only show mappings from concept scheme (URI or notation, multiple URIs/notations separated by `|`)

`to=[uriOrNotation1|uriOrNotation2|...]` specify the target URI or notation (truncated search possible by appending a `*`, multiple URIs/notations separated by `|`)

`toScheme=[uriOrNotation1|uriOrNotation2|...]` only show mappings to concept scheme (URI or notation, multiple URIs/notations separated by `|`)

`mode=[mode]` specify the mode for `from` / `fromScheme`, `to` / `toScheme`, and `identifier`, one of `and` (default) and `or`; note that 1) multiple values given for a single parameter are always connected via "or", and 2) `from` and `fromScheme` / `to` and `toScheme` are always connected via "and"

`direction=[direction]` specify the direction of the mapping. Available values are: `forward` (default), `backward` (essentially swaps `from` and `to`), `both` (combines forward and backward).

`type=[uri1|uri2|...]` only show mappings that conform to a certain type or types (see [JSKOS Concept Mappings](#)) (URIs separated by `|`)

`partOf=[uri1|uri2|...]` only show mappings that are part of certain concordances (URIs separated by `|`); value `none` returns mappings that are not part of a concordance, value `any` returns mappings that are part of any concordance

`creator=[string1|string2|...]` only show mappings that have a certain creator (separated by `|`)

`annotatedBy=[uri1|uri2|...]` has annotations by user with URI(s)

`annotatedFor=[motivation]` has annotations with a certain motivation (e.g. `assessing`); value `none` returns mappings that have no annotations at all, value `any` returns mappings that have any kind of annotation, values starting with `!` (e.g. `!assessing`) filter out annotations with that motivation. Note that to mitigate performance issues with negative assertions (`none` or `!xyz`), jskos-server will return the number 9999999 in the `x-Total-Count` header (see [this](#)).

`annotatedWith=[body]` has annotations with a certain body value (e.g. `+1`) OR has a sum of assessment annotations that conforms to the given comparison operation; for the latter, either `from` or `to` must be given, `annotatedFor` must be either not set or set to `assessing`, and the value of this parameter needs to consist of a comparison operator (`=`, `<`, `>`, `<=`, or `>=`) followed by a number. Example: `annotatedWith=>0` returns mappings with a positive assessment sum (equivalent to `annotatedWith=>=1`).

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting only `annotations` for mappings)

`download=[type]` returns the whole result as a download (available types are `json`, `ndjson`, `csv`, and `tsv`), ignores `limit` and `offset`; **note:** `csv` and `tsv` are restricted (and fixed) to 5 target concepts, meaning that if the data set includes a mapping with more than 5 target concepts, only the first 5 will appear in the export

`sort=[sort]` sorts by a specific field. Available are `created`, `modified`, and `mappingRelevance` (default). Results will always be additionally sorted by `from.memberSet.uri` and `_id` in order to create a stable and sensible sort.

`order=[order]` order to use for sorting. Available are `asc` and `desc` (default).

`cardinality=[cardinality]` cardinality of the mapping. Available are `1-to-n` (default) and `1-to-1`.

• Success Response

JSON array of [JSKOS Concept Mappings](#)

• Sample Call

```
curl https://coli-conc.gbv.de/api/mappings?from=http://dewey.info/class/612.116/e23/
```

```
[
  {
    "from": {
      "memberSet": [
        {
          "uri": "http://dewey.info/class/612.116/e23/",
          "notation": [
            "612.116"
          ]
        }
      ]
    },
    "to": {
      "memberSet": [
        {
          "uri": "http://rvk.uni-regensburg.de/nt/WW_8800-WW_8839",
          "notation": [
            "WW 8800-WW 8839"
          ]
        }
      ]
    },
    "fromScheme": {
      "uri": "http://bartoc.org/en/node/241",
      "notation": [
        "DDC"
      ]
    },
    "toScheme": {
      "uri": "http://bartoc.org/en/node/533",
      "notation": [
        "RVK"
      ]
    },
    "identifier": [
      "urn:jskos:mapping:content:fb92cbcd7466764dd2ca5fdf054bf55e65ec6b87",
      "urn:jskos:mapping:members:5aa92285bba839954baccdad7df5ef4558860ed"
    ],
    "@context": "https://gbv.github.io/jskos/context.json"
  }
]
```

GET /mappings/suggest

Suggests notations used in mappings.

• URL Params

`search=[notation]` specifies the notation (prefix) to search for

- **Success Response**

JSON array of suggestions in [OpenSearch Suggest Format](#).

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/mappings/suggest?search=A&limit=5
```



```
[
  "A",
  [
    "AN 74800",
    "AN 78950",
    "AN 70000",
    "AN 71000",
    "AN 96900"
  ],
  [
    42,
    25,
    19,
    18,
    17
  ],
  [],
  []
]
```



GET /mappings/voc

Lists all concept schemes used in mappings.

- **URL Params**

`from=[uri|notation]` restrict mappings to those from a concept

`to=[uri|notation]` restrict mappings to those to a concept

`mode=[mode]` specify the mode for `from` and `to`, one of `and` and `or` (default)

- **Success Response**

JSON array of [JSKOS Concept Schemes](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/mappings/voc?from=612.112&to=612.112
```



```
[
  {
    "uri": "http://bartoc.org/en/node/430",
    "notation": [
      "GND"
    ],
    "fromCount": 2
  },
  {
    "uri": "http://bartoc.org/en/node/241",
    "notation": [
      "DDC"
    ],
    "fromCount": 2,
    "toCount": 2
  },
  {
    "uri": "http://bartoc.org/en/node/533",
    "notation": [
      "RVK"
    ],
    "toCount": 2
  }
]
```



GET /mappings/infer

Returns mappings based on stored mappings and mappings derived by inference. If a request to [GET /mappings](#) results in stored mappings, only those are returned. If no stored mappings match the request, the following algorithm is applied to infer virtual mappings:

- Ancestors of the requested concept (`from`) are traversed from narrower to broader until matching mapping(s) from one of the ancestor concepts are found.
- The resulting mappings are filtered and transformed based on their mapping type:
 - `exactMatch` and `narrowMatch` result in `narrowMatch` (for instance *Optics* < *Sciences* when no mappings from *Optics* are stored but e.g. *Physics* is ancestor of *Optics* and mapped to *Sciences*)
 - `closeMatch` results in `narrowMatch` unless query parameter `strict` is set to a true value. In this case mappings of this type are ignored (for instance *Optics* < *Alchemy* when *Physics* is ancestor of *Optics* and mapped to *Alchemy* but this may lead to doubtful mappings such as *Computational Physics* < *Alchemy*)
 - `relatedMatch` and `mappingRelation` are not changed.

Inferred mappings don't have fields such as `uri`, `identifier`, `creator`, `created` ... but `uri` of the mapping used for inference is included in `source`.

Retrieval of ancestors depends on existence of parameter `fromScheme` (*this is experimental, details may change*):

- if missing, ancestors are searched for in the database (see [GET /concepts/ancestors](#))
- otherwise the concept scheme must exist in the database with a supported external API to query ancestors

• URL Params

This endpoint takes the same parameters as [GET /mappings](#), except that `to` and `download` are not allowed, `cardinality` can only be "1-to-1" and `direction` can only be "forward". An empty set is returned if parameter `from` is missing or if `from` is no URI and the concept scheme identified by `fromScheme` does not have a `namespace` to map `from` to an URI. In addition there are parameters:

`strict=[boolean]` values `1` or `true` disallow mapping type "closeMatch" for inferred mappings (default `false`)

`depth=[number]` a non-negative number of the depth used to infer mappings (not set by default); `0` means no inference, `1` means only the next ancestor concept (= broader) is used for inference, etc.

• Success Response

JSON array of [JSKOS Concept Mappings](#)

• Sample Call

```
curl https://coli-conc.gbv.de/api/mappings/infer?from=http%3A%2F%2Frvk.uni-regensburg.de%2Fnt%2FWI%25203130&fromS
```

```
[
  {
    "from": {
      "memberSet": [
        {
          "uri": "http://rvk.uni-regensburg.de/nt/WI%203130",
          "notation": [
            "WI 3130"
          ]
        }
      ]
    },
    "to": {
      "memberSet": [
        {
          "uri": "http://uri.gbv.de/terminology/bk/42.42",
          "notation": [
            "42.42"
          ]
        }
      ]
    },
    "fromScheme": {
      "uri": "http://bartoc.org/en/node/533",
      "notation": [
        "RVK"
      ]
    },
    "toScheme": {
      "uri": "http://bartoc.org/en/node/18785",
      "notation": [
        "BK"
      ]
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#narrowMatch"
    ],
    "source": [
      {
        "uri": "https://coli-conc.gbv.de/api/mappings/ef121206-a42d-4c3c-9ef3-b597c000acb4"
      }
    ],
    "identifier": [
      "urn:jskos:mapping:content:1b0fb2343795db4de7e1f8c7207b94a789614a15",
      "urn:jskos:mapping:members:2d22b62a0295959d587487d228d51836d05b1c50"
    ],
    "@context": "https://gbv.github.io/jskos/context.json"
  },
  {
    "from": {
      "memberSet": [
        {
          "uri": "http://rvk.uni-regensburg.de/nt/WI%203130",
          "notation": [
            "WI 3130"
          ]
        }
      ]
    },
    "to": {
      "memberSet": [
        {
          "uri": "http://uri.gbv.de/terminology/bk/42.44",
          "notation": [
            "42.44"
          ]
        }
      ]
    },
    "fromScheme": {
      "uri": "http://bartoc.org/en/node/533",
      "notation": [
        "RVK"
      ]
    },
    "toScheme": {
      "uri": "http://bartoc.org/en/node/18785",
      "notation": [
        "BK"
      ]
    }
  }
],
"....."
```

```
    "type": [
      "http://www.w3.org/2004/02/skos/core#narrowMatch"
    ],
    "source": [
      {
        "uri": "https://coli-conc.gbv.de/api/mappings/6b920456-db5d-49b1-a197-b851df6f9dbd",
      }
    ],
    "identifier": [
      "urn:jskos:mapping:content:8bb72e1605f9c25b0c97889439e6dde952e0cbd0",
      "urn:jskos:mapping:members:5870d87ec08c9a9a5ccba182bd96b92ad2f9d688"
    ],
    "@context": "https://gbv.github.io/jskos/context.json"
  }
]
```

GET /mappings/:_id

Returns a specific mapping.

- **URL Params**

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting only `annotations` for mappings)

- **Success Response**

JSKOS object for mapping.

- **Error Response**

If no mapping with `_id` could be found, it will return a 404 not found error.

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/mappings/5c450ba1a32a4a82d0f3fbf3
```



```

{
  "from": {
    "memberSet": [
      {
        "uri": "http://rvk.uni-regensburg.de/nt/TA-TD",
        "notation": [
          "TA - TD"
        ]
      }
    ]
  },
  "toScheme": {
    "template": "http://dewey.info/class/(.+)/e23/",
    "pattern": "[0-9][0-9]?|[0-9]{3}(-[0-9]{3})?|[0-9]{3}\\.[0-9]+(-[0-9]{3}\\.[0-9]+)?|[1-9][A-Z]?--[0-9]+|[1-9]",
    "uri": "http://bartoc.org/en/node/241",
    "notation": [
      "DDC"
    ]
  },
  "fromScheme": {
    "notation": [
      "RVK"
    ],
    "uri": "http://bartoc.org/en/node/533"
  },
  "to": {
    "memberSet": [
      {
        "uri": "http://dewey.info/class/500/e23/",
        "notation": [
          "500"
        ]
      }
    ]
  },
  "identifier": [
    "urn:jskos:mapping:content:d37d117b5e3d811447bc332b184ac6e5ac4bde6b",
    "urn:jskos:mapping:members:4c480744ea32e7e71ba39fae6cc8d8e4e0382912"
  ],
  "partOf": [
    {
      "uri": "http://coli-conc.gbv.de/concordances/rvk_ddc_ta-td"
    }
  ],
  "creator": [
    {
      "prefLabel": {
        "de": "GESIS"
      }
    }
  ],
  "url": "https://coli-conc.gbv.de/api/mappings/5c450ba1a32a4a82d0f3fbf3",
  "@context": "https://gbv.github.io/jskos/context.json"
}

```

POST /mappings

Saves a mapping or multiple mappings in the database. Mappings can be provided in JSKOS or SSSOM/TSV format. Mappings must include JSKOS fields `fromScheme` and `toScheme`. For JSKOS these fields can either be given with SSSOM/TSV metadata fields `subject_source` and `object_source` if URL parameter `scheme` is set to `given` or they can be inferred from concepts in the database with `scheme=lookup`.

• URL Params

`bulk=[boolean]` `1` or `true` enable bulk mode for importing multiple mappings into the database. Errors for individual mappings will be ignored and existing mappings will be overridden. The resulting set will only include the `id` for each mapping that was written into the database.

`scheme=given|lookup` whether take `fromScheme` / `toScheme` from passed data (`given` as default) or to look up concept URIs in the database (`lookup`).

• Success Response

JSKOS Mapping object as it was saved in the database, or array of mapping objects with only a `uri` if bulk mode was used..

• Error Response

When a single mapping is provided, an error can be returned if there's something wrong with it (see [errors](#)). When multiple mappings are provided, the first error will be returned, except if bulk mode is enabled in which errors for individual mappings are ignored.

Note that the `partOf` property is currently not allowed. Associating a mapping with a concordance has to be done in a separate [PUT](#) or [PATCH](#) request.

PUT /mappings/:_id

Overwrites a mapping in the database.

- **Success Reponse**

JSKOS Mapping object as it was saved in the database.

Note that any changes to the `created` property will be ignored. Note that changes to `partOf` (i.e. association with a concordance) are only possible if 1) `fromScheme` and `toScheme` are equal between the mapping and the concordance, 2) the authenticated user is creator of the mapping OR if the mapping is already part of a concordance, the user is creator/contributor of that concordance, and 3) the user is creator/contributor of the target concordance (if given).

PATCH /mappings/:_id

Adjusts a mapping in the database.

- **Success Reponse**

JSKOS Mapping object as it was saved in the database.

Note that any changes to the `created` property will be ignored. Note that changes to `partOf` (i.e. association with a concordance) are only possible if 1) `fromScheme` and `toScheme` are equal between the mapping and the concordance, 2) the authenticated user is creator of the mapping OR if the mapping is already part of a concordance, the user is creator/contributor of that concordance, and 3) the user is creator/contributor of the target concordance (if given).

DELETE /mappings/:_id

Deletes a mapping from the database.

- **Success Reponse**

Status 204, no content.

POST /mappings/apply

Apply mappings to a JSKOS set (*experimental*).

- **URL Params**

Same as [GET /mappings/infer](#) except `direction` can only be "forward", `cardinality` can only be "1-to-1", `mode` can only be "null" and the parameters `to`, `download`, `from`, `strict`, and `type` are not allowed.

Mappings can be selected with `fromScheme`, `toScheme`, `partOf`, `identifier`, `creator`, `annotatedBy`, `annotatedFor`, and `annotatedWith`. Inference can be selected with `depth`.

- **Success Reponse**

JSKOS set with additional concepts if these can be derived based on mappings. Each appended concept has fields:

- `uri` concept URI
- `inScheme` concept scheme of the concept
- `MAPPING` URI of the mapping that resulted in this enrichment

GET /voc

Lists supported terminologies (concept schemes).

- **URL Params**

`uri=[uri]` URIs for concept schemes separated by `|`. If `uri` is not given, all supported concept schemes are returned.

`type=URI` type URI to filter schemes

`languages=tag` language codes to filter schemes, separated by `,` (exact values). *Not to be confused with query parameter `language` at other endpoints!*

// Note: The `language` parameter at other endpoints means "give me labels in these languages". That's why it should have a different name here. Until then, it is removed.

`subject=URIs` subject URI(s) to filter schemes, separated by `|`

`license=URIs` license URI(s) to filter schemes, separated by `|`

`publisher=URI|label` publisher URI or label to filter schemes (only exact matches)

`partOf=URIs` filter by registry URI that is listed in `partOf` field of the scheme, separated by `|`

`sort=[property]` sort the results by a certain property. Possible values: `label` (preferred or alternative label in English or other languages), `notation` (string), `created` (timestamp), `modified` (timestamp), `counter` (number after last `/` in URI)

`order=[order]` order to use for sorting. Available are `asc` (default) and `desc`.

`notation` Notations for concept schemes separated by `|`. You can filter by one or more notations, i.e. `voc?notation=ASB|EuroVoc`

• Success Response

JSON array of [JSKOS Concept Schemes](#)

• Sample Call

```
curl https://coli-conc.gbv.de/api/voc?limit=1
```



```
[
  {
    "uri": "http://dewey.info/scheme/edition/e23/",
    "prefLabel": {
      "de": "Dewey-Dezimalklassifikation",
      "en": "Dewey Decimal Classification"
    },
    "notation": [
      "DDC"
    ],
    "identifier": [
      "http://bartoc.org/en/node/241"
    ],
    "license": [
      {
        "uri": "http://creativecommons.org/licenses/by-nc-nd/3.0/"
      }
    ],
    "publisher": [
      {
        "uri": "http://d-nb.info/gnd/1086052218",
        "prefLabel": {
          "de": "OCLC"
        },
        "altLabel": {
          "de": [
            "OCLC Online Computer Library Center"
          ]
        },
        "url": "https://www.oclc.org/"
      }
    ],
    "@context": "https://gbv.github.io/jskos/context.json"
  }
]
```



POST /voc

Saves a concept scheme or multiple concept schemes in the database. Each concept scheme has to have a unique `uri`.

• URL Params

`bulk=[boolean]` `1` or `true` enable bulk mode for importing multiple concept schemes into the database. Errors for individual concept schemes will be ignored and existing concept schemes will be overridden. The resulting set will only include the `id` for each concept scheme that was written into the database.

• Success Response

JSKOS Concept Scheme object or array as was saved in the database, or array of concept scheme objects with only a `uri` if bulk mode was used.

- **Error Response**

When a single concept scheme is provided, an error can be returned if there's something wrong with it (see [errors](#)). When multiple concept schemes are provided, the first error will be returned, except if bulk mode is enabled in which errors for individual concept schemes are ignored.

PUT /voc

Overwrites a concept scheme in the database. Is identified via its `uri` field.

- **Success Response**

JSKOS Concept Scheme object as it was saved in the database.

Note that any changes to the `created` property will be ignored.

DELETE /voc

Deletes a concept scheme from the database.

- **URL Params**

`uri=URI` URI for concept scheme to be deleted.

- **Success Response**

Status 204, no content.

GET /voc/top

Lists top concepts for a concept scheme.

- **URL Params**

`uri=[uri]` URI for a concept scheme

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors` and `narrower`)

- **Success Response**

JSON array of [JSKOS Concepts](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/voc/top?uri=http://dewey.info/scheme/edition/e23/
```



GET /voc/concepts

Lists concepts for a concept scheme.

- **URL Params**

`uri=[uri]` URI for a concept scheme (required)

`near=[latitude, longitude]` filters concepts by distance to a location (JSKOS field `location`), with `latitude` being a floating point value between -90 and 90 and `longitude` being a floating point value between -180 and 180

`distance=[distance]` value in kilometers which sets the radius for the `near` parameter if specified (default: 1)

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors` , `narrower` , and `annotations`); not supported for download

`download=[type]` returns the whole result as a download (available types are `json` and `ndjson`), ignores `limit` and `offset`

- **Success Response**

JSON array of [JSKOS Concepts](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/voc/concepts?uri=http://dewey.info/scheme/edition/e23/
```



DELETE /voc/concepts

Deletes all concepts of a certain concept scheme from the database.

- **URL Params**

`uri=URI` URI for a concept scheme

- **Success Response**

Status 204, no content.

GET /voc/suggest

Returns concept scheme suggestions.

- **URL Params**

`search=[keyword|notation]` specifies the keyword or notation (prefix) to search for

`language=[string]` comma-separated priority list of languages for labels in results

`format=[string]` return format for suggestions: `jskos` or `opensearch` (default)

- **Success Response**

JSON array of suggestions.

GET /voc/search

Currently the same as `/voc/suggest` with parameter `format=jskos`.

GET /concepts

Returns detailed data for concepts. Note that there is no certain order to the result set (but it should be consistent across requests). Note that no data is returned if no filtering parameter is given (one of `uri`, `notation`, `voc`, or `near`).

- **URL Params**

`uri=[uri]` URIs for concepts separated by `|`

`notation=[notation]` notations for concepts separated by `|`

`voc=[uri]` filter by concept scheme URI

`near=[latitude, longitude]` filters concepts by distance to a location (JSKOS field `location`), with `latitude` being a floating point value between -90 and 90 and `longitude` being a floating point value between -180 and 180

`distance=[distance]` value in kilometers which sets the radius for the `near` parameter if specified (default: 1)

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors`, `narrower`, and `annotations`); not supported for download

`download=[type]` returns the whole result as a download (available types are `json` and `ndjson`), ignores `limit` and `offset`

- **Success Response**

JSON array of [JSKOS Concepts](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/concepts?uri=http://dewey.info/class/612.112/e23/
```




```
[
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/612.11/e23/"
      }
    ],
    "created": "2000-02-02",
    "identifier": [
      "16d595ff-ec01-3e55-b425-016cf92bb950"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2013-12-04",
    "notation": [
      "612.112"
    ],
    "prefLabel": {
      "de": "Leukozyten (Weiße Blutkörperchen)"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/612.112/e23/",
    "narrower": [
      null
    ]
  }
]
```

POST /concepts

Saves a concept or multiple concepts in the database. Each concept has to have a unique `uri` as well as a concept scheme that is available on the server in the `inScheme` or `topConceptOf` field.

• URL Params

`bulk=[boolean]` `1` or `true` enable bulk mode for importing multiple concepts into the database. Errors for individual concepts will be ignored and existing concepts will be overridden. The resulting set will only include the URI for each concept that was written into the database.

• Success Reponse

JSKOS Concept object or array as was saved in the database, or array of concept objects with only a URI if bulk mode was used.

• Error Response

When a single concept is provided, an error can be returned if there's something wrong with it (see [errors](#)). When multiple concepts are provided, the first error will be returned, except if bulk mode is enabled in which errors for individual concepts are ignored.

PUT /concepts

Overwrites a concept in the database. Is identified via its `uri` field.

• Success Reponse

JSKOS Concept object as it was saved in the database.

DELETE /concepts

Deletes a concept from the database.

• URL Params

`uri=URI` URI for concept to be deleted.

• Success Reponse

Status 204, no content.

GET /concepts/narrower

Returns narrower concepts for a concept.

Note: The old `/narrower` endpoint is deprecated as of version 2.0 and will be removed in version 3.0.

- **URL Params**

`uri=[uri]` URI for a concept

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors` and `narrower`)

- **Success Response**

JSON array of [JSKOS Concepts](#)

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/concepts/narrower?uri=http://dewey.info/class/612.112/e23/
```



```
[
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/612.112/e23/"
      }
    ],
    "created": "2000-02-02",
    "identifier": [
      "cf6faa73-e5e7-3856-9429-611a8a39d253"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2005-11-02",
    "notation": [
      "612.1121"
    ],
    "prefLabel": {
      "de": "Biochemie"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/612.1121/e23/",
    "narrower": []
  },
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/612.112/e23/"
      }
    ],
    "created": "2000-02-02",
    "http://www.w3.org/2002/07/owl#deprecated": true,
    "identifier": [
      "23519115-b023-3812-a2c1-6fc99e169ae3"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2005-11-02",
    "notation": [
      "612.1122"
    ],
    "prefLabel": {
      "de": "Biophysik"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/612.1122/e23/",
    "narrower": []
  },
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/612.112/e23/"
      }
    ],
    "created": "2000-02-02",
    "identifier": [
      "4a070e77-094c-3638-9067-2b3625d612e9"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2005-11-02",
    "notation": [
      "612.1127"
    ],
    "prefLabel": {
      "de": "Biochemie und Biochemie"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/612.1127/e23/",
    "narrower": []
  }
]
```



```

      "de": "Anzahl und Ausanzlung"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/612.1127/e23/",
    "narrower": []
  }
]

```

GET /concepts/ancestors

Returns ancestor concepts for a concept.

Note: The old `/ancestors` endpoint is deprecated as of version 2.0 and will be removed in version 3.0.

• URL Params

`uri=[uri]` URI for a concept

`properties=[list]` with `[list]` being a comma-separated list of properties (currently supporting `ancestors` and `narrower`)

• Success Response

JSON array of [JSKOS Concepts](#)

• Sample Call

```
curl https://coli-conc.gbv.de/api/concepts/ancestors?uri=http://dewey.info/class/61/e23/
```



```

[
  {
    "@context": "https://gbv.github.io/jskos/context.json",
    "created": "2000-02-02",
    "identifier": [
      "856c92e9-8b1f-3131-bfbc-f2d2266527d3"
    ],
    "modified": "2005-11-02",
    "notation": [
      "6"
    ],
    "prefLabel": {
      "de": "Technik, Medizin, angewandte Wissenschaften"
    },
    "topConceptOf": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/6/e23/",
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "narrower": [
      null
    ]
  }
]

```



GET /concepts/suggest

Returns concept suggestions.

Note: The old `/suggest` endpoint is deprecated as of version 2.0 and will be removed in version 3.0.

• URL Params

`search=[keyword|notation]` specifies the keyword or notation (prefix) to search for

`language=[string]` comma-separated priority list of languages for labels in results

`format=[string]` return format for suggestions: `jskos` or `opensearch` (default)

- **Success Response**

JSON array of suggestions.

- **Sample Calls**

```
curl https://coli-conc.gbv.de/api/concepts/suggest?search=Krebs&limit=5
```



```
[
  "Krebs",
  [
    "133.5265 Krebs",
    "639.5 Krebstierfang",
    "639.6 Krebstierzucht",
    "616.994 Krebserkrankungen",
    "641.695 Krebstiere"
  ],
  [
    "",
    "",
    "",
    "",
    ""
  ],
  [
    "http://dewey.info/class/133.5265/e23/",
    "http://dewey.info/class/639.5/e23/",
    "http://dewey.info/class/639.6/e23/",
    "http://dewey.info/class/616.994/e23/",
    "http://dewey.info/class/641.695/e23/"
  ]
]
```



```
curl https://coli-conc.gbv.de/api/concepts/suggest?search=Krebs&limit=2&format=jskos
```



```
[
  {
    "_id": "http://dewey.info/class/133.5265/e23/",
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/133.526/e23/"
      }
    ],
    "created": "2000-02-02",
    "identifier": [
      "57e89e64-9de0-35c1-88da-856529d547c8"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2005-11-02",
    "notation": [
      "133.5265"
    ],
    "prefLabel": {
      "de": "Krebs"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/133.5265/e23/",
    "narrower": [],
    "priority": 292
  },
  {
    "_id": "http://dewey.info/class/639.5/e23/",
    "@context": "https://gbv.github.io/jskos/context.json",
    "broader": [
      {
        "uri": "http://dewey.info/class/639/e23/"
      }
    ],
    "created": "2000-02-02",
    "identifier": [
      "8b1dc20e-5d1e-34f4-8478-3fa022ba6fe0"
    ],
    "inScheme": [
      {
        "uri": "http://dewey.info/scheme/edition/e23/"
      }
    ],
    "modified": "2005-11-02",
    "notation": [
      "639.5"
    ],
    "prefLabel": {
      "de": "Krebstierfang"
    },
    "type": [
      "http://www.w3.org/2004/02/skos/core#Concept"
    ],
    "uri": "http://dewey.info/class/639.5/e23/",
    "narrower": [
      null
    ],
    "priority": 195
  }
]
```

GET /concepts/search

Currently the same as `/concepts/suggest` with parameter `format=jskos`. Additionally, search supports the parameter `properties=[list]` as in the other concept methods.

Note: The old `/search` endpoint is deprecated as of version 2.0 and will be removed in version 3.0.

GET /registries

Returns an array of registries. Each registry has a property `id` under which the specific registry can be accessed.

- **URL Params**

`id=[id]` specify a registry ID

- **Success Response**

Array of registries in [JSKOS Registry](#) format

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/registries?limit=1
```

```
[
  {
    "API": [
      {
        "type": "http://bartoc.org/api-type/jskos",
        "url": "https://api.dante.gbv.de/"
      }
    ],
    "startDate": "2017",
    "definition": {
      "en": [
        "DANTE (Datendrehscheibe für Normdaten und Terminologien) is a vocabulary server hosted by VZG based on e"
      ]
    },
    "prefLabel": {
      "en": "DANTE"
    },
    "type": [
      "http://www.w3.org/ns/dcat#Catalog",
      "http://bartoc.org/full-repository"
    ],
    "uri": "http://bartoc.org/en/node/19999",
    "url": "https://api.dante.gbv.de/"
  }
]
```

GET /registries/suggest

Returns registry suggestions.

- **URL Params**

`search=[keyword|notation]` specifies the keyword or notation (prefix) to search for

- **Success Response**

JSON array of suggestions.

- **Sample Calls**

```
curl https://coli-conc.gbv.de/api/registries/suggest?search=Stanford
```

```
[
  "Stanford",
  [
    "ERMS VEST Registry"
  ],
  [
    "",
    "",
    ""
  ],
  [
    "http://bartoc.org/en/node/18927"
  ]
]
```

```
curl https://coli-conc.gbv.de/api/concepts/suggest?search=Krebs&limit=2&format=jskos
```

POST /registries

Saves one registry or multiple registries in the database.

- **URL Params**

`bulk=[boolean]` `1` or `true` enable bulk mode for importing multiple registries into the database. Existing registries will be overridden. The resulting set will only include the `id` for each registry that was written into the database.

- **Success Response**

Registry object or array of object as was saved in the database in [JSKOS Registry](#) format, or array of registry objects with only a `id` if bulk mode was used.

- **Error Response**

Bulk mode: invalid registries are skipped. Non-bulk mode: the first error is thrown (see [errors](#)).

PUT /registries

Overwrites a registry in the database, identified by its `uri` field.

- **Success Response**

Registry object as it was saved in the database in [JSKOS Registry](#) format.

PATCH /registries

Modifies a registry in the database, identified by its `uri` field.

- **Success Response**

Registry object as it was saved in the database in [JSKOS Registry](#) format.

DELETE /registries

Deletes a registry from the database.

- **URL Params**

`uri=URI` URI for registry to be deleted.

- **Success Response**

Status 204, no content.

GET /annotations

Returns an array of annotations. Each annotation has a property `id` under which the specific annotation can be accessed.

- **URL Params**

`id=[id]` specify an annotation ID

`creator=[uriOrName]` only return annotations that have a certain creator (name or URI) (separated by `|`)

`target=[target]` only return annotations with a specific target URI (e.g. a mapping URI)

`bodyValue=[bodyValue]` only return annotations with a specific bodyValue (e.g. `+1` , `-1`)

`motivation=[motivation]` only return annotations with a specific motivation (e.g. `assessing` , `moderating` , `tagging`)

`download=[type]` returns the whole result as a download (available types are `json` and `ndjson`), ignores `limit` and `offset`

- **Success Response**

Array of annotations in [Web Annotation Data Model](#) format

Releases 66 • **Sample Call**

 **JSKOS Server** last week `curl https://coli-conc.gbv.de/api/annotations?bodyValue=+1&limit=1`



[+ 65 releases](#)

Deployments 66

 **github-pages** last week

[+ 65 deployments](#)

Packages 1	[
 jskos-server	{
	"target": {
	"id": "https://coli-conc.gbv.de/api/mappings/f8eff4e2-a6df-4d2c-8382-523072c59af7"
	},
Contributors	"motivation": "assessing",
	"bodyValue": "+1",
  	"creator": "https://orcid.org/0000-0002-4087-8227",
	"created": "2019-01-31T09:44:12.699Z",
	"id": "https://coli-conc.gbv.de/api/annotations/2575e276-29c6-4d36-8477-b21be1790e64",
	"@context": "http://www.w3.org/ns/anno.jsonld",
Languages	"type": "Annotation"
	}
● JavaScript 96.8	]

GET /annotations/:_id

Returns a specific annotation.

- **Success Response**

Object for annotation in [Web Annotation Data Model](#) format.

- **Error Response**

If no annotation with `_id` could be found, it will return a 404 not found error.

- **Sample Call**

```
curl https://coli-conc.gbv.de/api/annotations/5f23368f-a63b-4b69-acd6-b403110df97c
```

```
{
  "target": {
    "id": "https://coli-conc.gbv.de/api/mappings/f0cc5f65-5712-4820-9638-e662c0c4314e"
  },
  "motivation": "assessing",
  "bodyValue": "+1",
  "creator": {
    "id": "https://coli-conc.gbv.de/login/users/722cc9c5-2ce3-4ca0-b4fb-fef1f62236af",
    "name": "Jakob Voß"
  },
  "created": "2019-03-11T09:11:10.665Z",
  "id": "https://coli-conc.gbv.de/api/annotations/5f23368f-a63b-4b69-acd6-b403110df97c",
  "@context": "http://www.w3.org/ns/anno.jsonld",
  "type": "Annotation"
}
```

POST /annotations

Saves an annotation or multiple annotations in the database.

- **URL Params**

`bulk=[boolean]` 1 or true enable bulk mode for importing multiple annotations into the database. Errors for individual annotations will be ignored and existing annotations will be overridden. The resulting set will only include the `id` for each annotation that was written into the database.

- **Success Response**

Annotation object or array of object as was saved in the database in [Web Annotation Data Model](#) format, or array of annotation objects with only a `id` if bulk mode was used.

- **Error Response**

When a single annotation is provided, an error can be returned if there's something wrong with it (see [errors](#)). When multiple annotations are provided, the first error will be returned, except if bulk mode is enabled in which errors for individual annotations are ignored.

PUT /annotations/:_id

Overwrites an annotation in the database.

- **Success Response**

Annotation object as it was saved in the database in [Web Annotation Data Model](#) format.

Note that any changes to the `created` property will be ignored.

PATCH /annotations/:_id

Adjusts an annotation in the database.

- **Success Reponse**

Annotation object as it was saved in the database in [Web Annotation Data Model](#) format.

Note that any changes to the `created` property will be ignored.

DELETE /annotations/:_id

Deletes an annotation from the database.

- **Success Reponse** Status 204, no content.

Change Stream Endpoints

JSKOS-Server provides WebSocket endpoints that push live notifications whenever data changes. Available routes:

- `/voc/changes` — broadcasts events for **ConceptSchemes**
- `/concepts/changes` — broadcasts events for **Concepts**
- `/mappings/changes` — broadcasts events for **Mappings**
- `/concordances/changes` — broadcasts events for **Concordances**
- `/annotations/changes` — broadcasts events for **Annotations**
- `/registries/changes` — broadcasts events for **Registries**

Connection to websocket

```
# Example for ConceptSchemes
wscat -c ws://<host>:<port>/voc/changes
```



Messages

Each message is a JSON object with the following fields:

Field	Type	Description
<code>objectType</code>	<code>string</code>	The JSKOS object type. One of: <code>ConceptScheme</code> , <code>Concept</code> , <code>ConceptMapping</code> , <code>Concordance</code> , <code>Annotation</code> , <code>Registry</code> .
<code>type</code>	<code>string</code>	Change type, derived from the MongoDB operation: • <code>insert</code> → <code>create</code> • <code>update</code> / <code>replace</code> → <code>update</code> • <code>delete</code> → <code>delete</code>
<code>id</code>	<code>string</code> or <code>ObjectId</code>	The <code>_id</code> of the changed MongoDB document.
<code>timestamp</code>	<code>string</code>	The ISO 8601 timestamp of the change event, derived from the MongoDB change stream <code>clusterTime</code> (available since MongoDB 3.6).
<code>document</code>	<code>object</code> (<i>optional</i>)	The full JSKOS record as stored in MongoDB. Present only for <code>create</code> and <code>update</code> .

► Example: `create` event

► Example: `delete` event

Errors

If possible, errors will be returned as a JSON object in the following format (example):

```
{
  "error": "EntityNotFoundError",
  "status": 404,
  "message": "The requested entity ABC could not be found.",
}
```



Validation errors reported from [/validate endpoint](#) are no API errors (unless validation could not be executed for some reason).

The following errors are currently caught and returned as JSON:

EntityNotFoundError

Status code 404. Will be returned if `GET /mappings/:_id` or `GET /annotations/:_id` are requested with an unknown ID.

MalformedBodyError

Status code 400. Will be returned for `POST` / `PUT` / `PATCH` if the body was not JSON or missing.

MalformedRequestError

Status code 400. Will be returned if a required parameter is missing (currently implemented in `GET /.../:_id` endpoints, but should not be possible to reach).

DuplicateEntityError

Status code 422. Will be returned for `POST` if an entity with the same ID/URI already exists in the database.

InvalidBodyError

Status code 422. Will be returned for `POST` / `PUT` / `PATCH` if the body was valid JSON, but could not be validated (e.g. does not pass the JSKOS Schema).

CreatorDoesNotMatchError

Status code 403. Will be returned by `PUT` / `PATCH` / `DELETE` endpoints if the authenticated creator does not match the creator of the entity that is being edited.

BackendError

Status code 500. Will be returned if there's a backend error not related to the database or configuration.

DatabaseAccessError

Status code 500. Will be returned if the database is not available or if the current database request failed with an unknown error.

DatabaseInconsistencyError

Status code 500. Will be returned if there is an inconsistency issue with our database. Please [contact us](#) with the full error message if this occurs!

ConfigurationError

Status code 500. Will be returned if there is an error in the configuration that prevents the application from working correctly.

ForbiddenAccessError

Status code 403. Will be returned if the user is not allow access (i.e. when not on the whitelist or when an identity provider is missing).

Deployment

The application is currently deployed at <http://coli-conc.gbv.de/api/>. At the moment, there is no automatic deployment of new versions

Notes about depolyment on Ubuntu

It is recommended to use a [newer version of Node.js](#). Installing the dependencies might also require installing nodejs-legacy: `sudo apt-get install nodejs-legacy` ([more info here](#)). One possibility for running the application in production on Ubuntu 16.04 is described [here](#). (Information about restarting pm2-based services on system reboot [here](#).)

Update an instances deployed with PM2

```
# get updates from repository
git pull

# install dependencies
npm ci

# restart the process (adjust process name if needed)
pm2 restart jskos-server
```



Daily Import

If you'd like to run the import script daily to refresh current mappings, you can for example use a cronjob:


```
# Runs import script for jskos-server in /srv/cocoda/jskos-server at 1 AM each day.  
00 01 * * * cd /srv/cocoda/jskos-server; ./scripts/import.sh
```



Running Behind a Reverse Proxy

There are certain things to consider when running `jskos-server` behind a reverse proxy:

1. Make sure the base URL is configured correctly in `config.json` so that correct URIs will be generated. Test this by creating a new mapping and making sure the URI of that mapping is correct and accessible.
2. Provide a list of trusted proxy IPs or ranges in the `proxies` key in `config.json`. E.g. `"proxies": ["123.456.789.101", "234.567.891.011"]`. See also: [Express behind proxies](#).

3. The reverse proxy should be configured so that the base URL has a trailing slash: ~~https://example.com/api~~ ❌ - https://example.com/api/ ✅ (Note: Not implementing this has no further consequences except that /api will not be accessible.)
4. The reverse proxy should also be configured so that any URL **except** the base URL has **no** trailing slash: ~~https://example.com/api/status~~ ❌ - https://example.com/api/status ✅
5. Make sure the target parameter (i.e. the actual IP and port where jskos-server is running) has a trailing slash.
6. Make sure the proxy is configured to correctly set the X-Forwarded-For header.

The following would be an example for 2./3./4. with an Apache reverse proxy:

```
<VirtualHost *:8099>
  Define API_PATH /api
  ServerName example.com

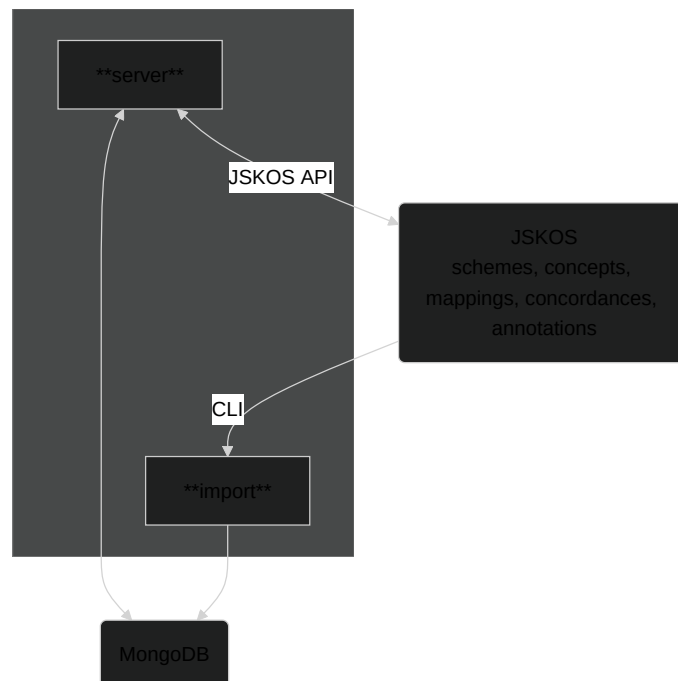
  RewriteEngine on
  # Remove trailing slash from everything EXCEPT the base URL
  RewriteCond %{REQUEST_URI} !^${API_PATH}/$
  RewriteRule ^(.+)/+$ $1 [R=301,L]
  # Force trailing slash for base URL only
  RewriteCond %{REQUEST_URI} ^${API_PATH}$
  RewriteRule ^(.+[^/])$ %{REQUEST_URI}/ [R=301,L]

  # Forward to jskos-server
  ProxyPass ${API_PATH}/ http://127.0.0.1:3000/
  ProxyPassReverse ${API_PATH}/ http://127.0.0.1:3000/

  # ...
</VirtualHost>
```

Data flow

JSKOS data is internally stored in a MongoDB database. The data can be read and written via [JSKOS API](#) and imported via local command line import script. In addition there are helper scripts not shown in the diagram.



Related works

jskos-server is developed together with the [cocoda](#) mapping application.

Alternative open source applications for hosting concept schemes are listed at <https://bartoc.org/software>.

See [cocoda-sdk](#) for efforts to provide uniform access to vocabulary information from different applications and sources.

Maintainers

- [@nichtich](#)

Contribute

PRs accepted against the `dev` branch.

Small note: If editing the README, please conform to the [standard-readme](#) specification.

Publish

For maintainers only

Never work on the main branch directly. Always make changes on a feature branch for specific issues or on the `dev` branch for small fixes and then run the release script:

```
npm run release:patch # or minor or major
```



License

MIT © 2018 Verbundzentrale des GBV (VZG)

